LAPORAN PRAKTIKUM

ALGORITMA PEMROGRAMAN 2

MODUL 5 – REKURSIF

NAMA : Adib Hanan Kautsar

NIM : 109082500182

KELAS S1IF-13-06

PROGRAM STUDI TEKNIK INFORMATIKA

FAKULTAS INFORMATIKA

TELKOM UNIVERSITY PURWOKERTO

2026

A. Dasar Teori

Rekursif adalah sebuah teknik dalam pemrograman di mana suatu subprogram, baik berupa fungsi maupun prosedur, memanggil dirinya sendiri untuk menyelesaikan suatu masalah. Teknik ini bekerja dengan cara memecah masalah utama menjadi sub masalah yang lebih kecil dan identik, sehingga solusi dapat ditemukan melalui penyelesaian unit-unit kecil tersebut secara berulang. Dalam implementasinya, rekursif sering kali menjadi alternatif yang lebih elegan dibandingkan dengan struktur kontrol perulangan iteratif (seperti for loop) untuk menangani kasus-kasus matematika atau struktur data tertentu.

Komponen paling krusial dalam algoritma rekursif adalah base case atau basis. Base case merupakan kondisi berhenti yang mencegah terjadinya pemanggilan fungsi tanpa henti (infinite recursion). Tanpa adanya base case yang jelas, program akan terus mengeksekusi dirinya sendiri hingga menyebabkan stack overflow atau penggunaan memori yang berlebihan. Oleh karena itu, menentukan kondisi kapan rekursi harus berhenti adalah langkah pertama dan terpenting dalam merancang program rekursif.

Selain base case, komponen kedua adalah recursive case atau kasus rekursif. Recursive case adalah bagian di mana subprogram memanggil dirinya sendiri dengan parameter yang biasanya mendekati kondisi base case. Kondisi ini umumnya merupakan negasi atau komplemen dari kondisi basis. Sebagai contoh, jika basisnya adalah ketika variabel bernilai 1, maka kasus rekursifnya adalah selama variabel tersebut belum bernilai 1.

Proses eksekusi rekursif melibatkan dua fase utama, yaitu forward dan backward. Fase forward terjadi saat program terus melakukan pemanggilan ke dalam dirinya sendiri hingga mencapai titik base case. Setelah kondisi basis terpenuhi, program akan melakukan fase backward, yaitu proses kembali ke pemanggil sebelumnya untuk menyelesaikan instruksi yang tersisa di dalam stack memori. Pemahaman terhadap alur maju mundur ini sangat penting karena penempatan baris kode (sebelum atau sesudah pemanggilan rekursif) akan sangat memengaruhi urutan keluaran (output) program

B. Guided

1. [guided1]

```go
package main
 import "fmt"
func main() {
var n int
fmt.Scan(&n)
baris (n)
}
```

```go
 func baris (bilangan
int) {     if bilangan ==
1 {
fmt.Print("1")
    } else {
fmt.Print(bilangan)
baris (bilangan - 1)
    }
}
```

Output :

```
guided> go run soal1.go
10]
10987654321
```

Penjelasan : fungsi akan terus mencetak angka dan memanggil dirinya sendiri dengan nilai yang berkurang 1 hingga mencapai angka 1 (base-case).

2. [guided2]

```go
package main import "fmt"
func main() {     var n int
fmt.Scan(&n)
fmt.Println(penjumlahan (n))
}  func penjumlahan (n int)
int {     if n == 1 {
return 1     } else {
        return n + penjumlahan (n - 1)
    }
}
```

Output :

```
guided> go run soal2.go
10
55
```

Penjelasan :
Setiap pemanggilan fungsi akan "menabung" nilai n dan menjumlahkannya dengan hasil dari pemanggilan berikutnya hingga berhenti di angka 1).

3. [guided3]

```go
package main import "fmt"
func main() {     var n
int     fmt.Scan(&n)
fmt.Println(pangkat (n))
}  func pangkat (n int) int {
if n == 1 {         return 1
}else {         return 2 *
pangkat (n - 1)
    }
}
```

Output :

```
guided> go run soal3.go
10
512
```

Penjelasan :
Rekursi dilakukan sebanyak n kali, di mana setiap tahap mengalikan angka 2 sampai mencapai pangkat 0.

C. Unguided

1. Soal Unguided 1

Deret fibonacci adalah sebuah deret dengan nilai suku ke-0 dan ke-1 adalah 0 dan 1, dan nilai suku ke-n selanjutnya adalah hasil penjumlahan dua suku sebelumnya. Secara umum dapat diformulasikan $S_n = S_{n-1} + S_{n-2}$. Berikut ini adalah contoh nilai deret fibonacci hingga suku ke-10. Buatlah program yang mengimplementasikan fungsi rekursif pada deret fibonacci tersebut.

n	0	1	2	3	4	5	6	7	8	9	10
S_n	0	1	1	2	3	5	8	13	21	34	55

Jawaban:

```
package main
 import
"fmt"
 func main() {     var n
int     fmt.Scan(&n)
fmt.Println(fibonacci(n))
} func fibonacci(n int)
int {     if n == 0 {
return 0      } else if n
== 1 {         return 1
    }     return fibonacci(n-1) +
fibonacci(n-2)  }
```

Output :

```
PS C:\Users\LENOVO\Desktop\109082500182_Adib-Hanan-Kautsar\Week 5\
unguided> go run 1.go
2
1
```

Penjelasan :

Fungsi memanggil dirinya sendiri dua kali untuk mendapatkan jumlah dari dua suku sebelumnya sesuai formula.

2. Soal Unguided 3

Buatlah program yang mengimplementasikan rekursif untuk menampilkan faktor bilangan dari suatu N, atau bilangan yang apa saja yang habis membagi N.

Masukan terdiri dari sebuah bilangan bulat positif N.

Keluaran terdiri dari barisan bilangan yang menjadi faktor dari N (terurut dari 1 hingga N ya).

Jawaban :

```go
package main
 import "fmt"
func main() {
var n int
fmt.Scan(&n)
```

```go
    cariFaktor(n, 1)
}  func cariFaktor(n, i
int) {     if i > n {
return
    }     if n %
i == 0 {
        fmt.Printf("%d ", i)
    }
cariFaktor(n, i+1)
}
```

Output :

Penjelasan :

Fungsi mengecek setiap angka dari 1 sampai N. Jika N habis dibagi angka tersebut, maka angka itu adalah faktornya.

3. Soal Unguided 5

Buatlah program yang mengimplementasikan rekursif untuk menampilkan barisan bilangan ganjil.

Masukan terdiri dari sebuah bilangan bulat positif N.

Keluaran terdiri dari barisan bilangan ganjil dari 1 hingga N.

Jawaban :

```go
package main
import "fmt"
func main() {
var n int
fmt.Scan(&n)
ganjil(n, 1)
} func ganjil(n, i
int) {     if i > n {
return
    }     if i %
2 != 0 {
        fmt.Printf("%d ", i)
    }
ganjil(n, i+1)
}
```

Output :

Penjelasan :

Program melakukan pengecekan angka secara berurutan dan hanya mencetak angka yang memiliki sisa bagi tidak nol (ganjil) hingga mencapai batas N.

D. Kesimpulan

Rekursif adalah teknik pemrograman yang kuat untuk menyelesaikan masalah dengan pendekatan pemecahan sub masalah yang identik. Dari hasil praktikum, dapat disimpulkan bahwa setiap algoritma rekursif wajib memiliki base case sebagai syarat mutlak agar program tidak berjalan selamanya. Selain itu, efektivitas rekursi sangat bergantung pada pemahaman alur forward dan backward, di mana urutan eksekusi sangat menentukan hasil akhir keluaran program. Meskipun setiap rekursif memiliki padanan iteratif, teknik ini memberikan kemudahan dalam penulisan kode untuk struktur yang sifatnya berulang secara matematis seperti faktorial atau Fibonacci.